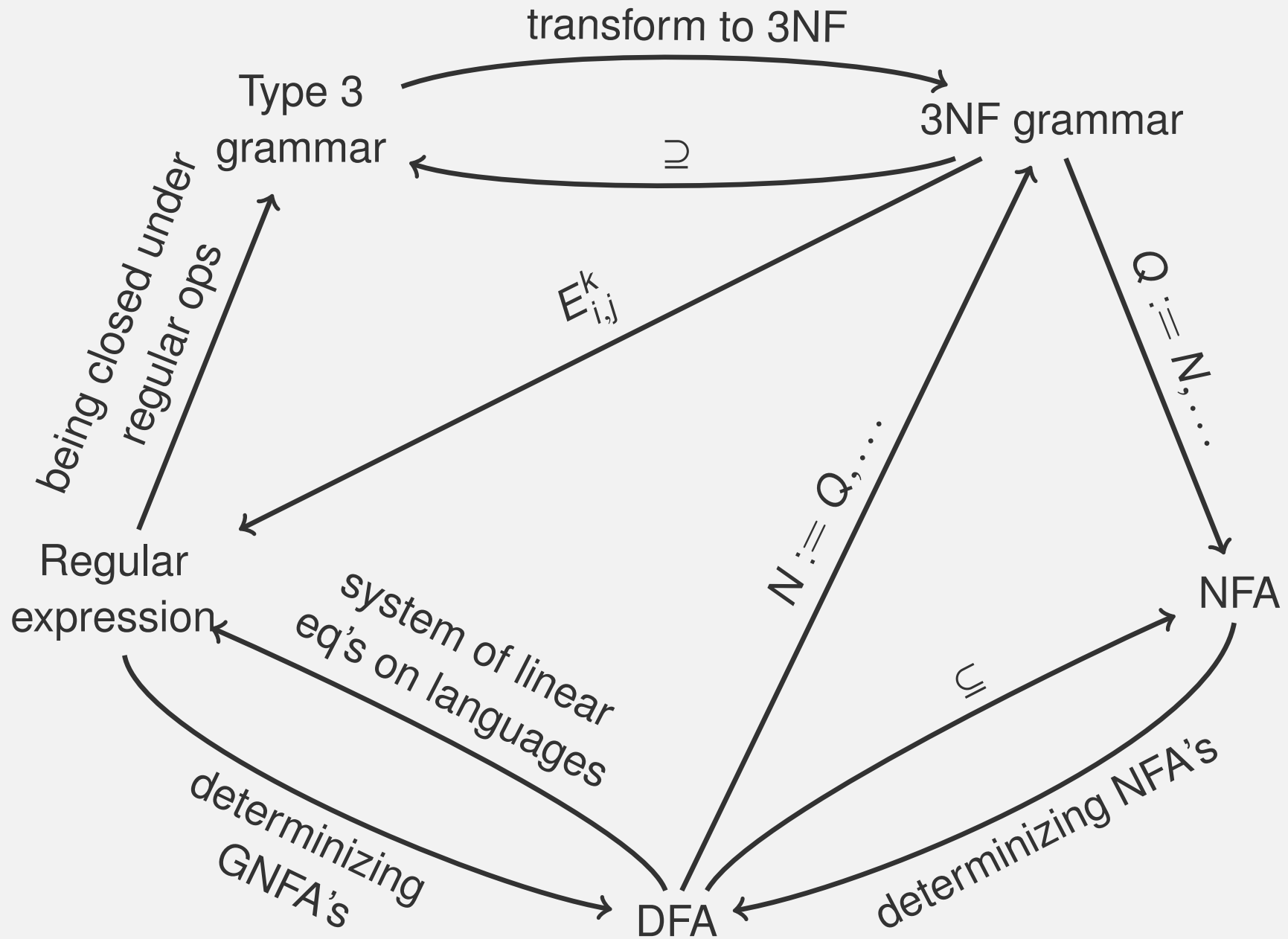


Fundamentals of theory of computation I.

Lecture 7

Krisztián Tichler
ktichler@inf.elte.hu

Formal tools describing $\mathcal{L}_3$



Solving a system of linear equations on languages

(Reminder:)

Solving a system of linear equations on languages

Given two regular expressions R and Q . Consider the problem of determining a regular expression P satisfying the equation $P = R + P \cdot Q$. It has a solution $P = R \cdot Q^*$ and this solution is unique if $\varepsilon \notin L(Q)$ holds.

A generalization of this theorem is the following (same proof).

Arden's Theorem

Given two languages L_1 and L_2 . Consider the problem of determining an unknown language X satisfying the equation $X = L_1 X \cup L_2$. It has a solution $X = L_1^* L_2$ and this solution is unique if $\varepsilon \notin L_1$ holds.

According to the theorem on $\mathcal{L}_i$ being closed under regular operations if L_1 and L_2 are type i languages then $X \in \mathcal{L}_i$ ($i \in \{0, 1, 2, 3\}$) holds.

Solving a system of linear equations on languages

Theorem

Let $\mathbf{x} = \mathbf{M}\mathbf{x} \cup \mathbf{v}$ be a system of linear equations on languages,

where a matrix of languages $\mathbf{M} = \begin{bmatrix} L_{11} & \cdots & L_{1n} \\ \vdots & \ddots & \vdots \\ L_{n1} & \cdots & L_{nn} \end{bmatrix}$ and a vector of

languages $\mathbf{v} = \begin{bmatrix} L_1 \\ \vdots \\ L_n \end{bmatrix}$ is given and an unknown vector of languages

$\mathbf{x} = \begin{bmatrix} X_1 \\ \vdots \\ X_n \end{bmatrix}$ is to be determined. If $\mathcal{L} = \bigcup_{j=1}^n \bigcup_{k=1}^n \{L_{jk}\} \cup \bigcup_{j=1}^n \{L_j\} \subseteq \mathcal{L}_i$

($i \in \{0, 1, 2, 3\}$) and $\varepsilon \notin L_{jk}$ ($1 \leq j \leq n, 1 \leq k \leq n$) then there is a unique solution from $\mathcal{L}_i$, and the solution can be expressed by finite application of regular operations on some languages of $\mathcal{L}$.

Remark: So, the j th equation is $X_j = L_{j1}X_1 \cup \cdots \cup L_{jn}X_n \cup L_j$.

Solving a system of linear equations on languages

Proof (sketch):

We reduce the number of unknowns and equations by Gauss elimination.

Last equation: $X_n = L_{nn}X_n \cup \bigcup_{k=1}^{n-1} L_{nk}X_k \cup L_n$.

According to Arden's theorem we have $X_n = L_{nn}^*(\bigcup_{k=1}^{n-1} L_{nk}X_k \cup L_n)$ implying a system of equations $\mathbf{x}' = \mathbf{M}'\mathbf{x}' \cup \mathbf{v}'$ for unknown

languages $\mathbf{x}' = \begin{bmatrix} X_1 \\ \vdots \\ X_{n-1} \end{bmatrix}$, a matrix of languages

$$\mathbf{M}' = \begin{bmatrix} L'_{11} & \cdots & L'_{1(n-1)} \\ \vdots & \ddots & \vdots \\ L'_{(n-1)1} & \cdots & L'_{(n-1)(n-1)} \end{bmatrix} \text{ and vector } \mathbf{v}' = \begin{bmatrix} L'_1 \\ \vdots \\ L'_{n-1} \end{bmatrix}.$$

where $L'_{jk} = L_{jk} \cup L_{jn}L_{nn}^*L_{nk}$ and $L'_j = L_j \cup L_{jn}L_{nn}^*L_n$.

Solving a system of linear equations on languages

We have one less unknowns and one less equations. One can check that $\varepsilon \notin L'_{jk}$ and $L'_{jk} \in \mathcal{L}_i$ ($\mathcal{L}_i$ is closed under regular operations).

Continuing Gauss elimination, in the end, we get a single equation with a single unknown. This equation can be solved by Arden's Theorem. Other unknowns can be determined by substitutions.

The claim on uniqueness follows from the statement on uniqueness in Arden's theorem and an inductive argument proving ε is not an element of any language from the coefficient matrices during the process of Gauss elimination.

Solving a system of linear equations on languages

Example:

Solve the system of linear equations.

$$X = a^*bX + b^*aY + ba$$

$$Y = b^*aX + a^*bY + ab$$

Solution:

X can be expressed as a function of Y by Arden's Theorem.

$$X = (a^*b)^*(ba + b^*aY).$$

$$Y = b^*a(a^*b)^*(ba + b^*aY) + a^*bY + ab$$

$$Y = (b^*a(a^*b)^*b^*a + a^*b)Y + (b^*a(a^*b)^*ba + ab), \text{ implying}$$

$$Y = (b^*a(a^*b)^*b^*a + a^*b)^*(b^*a(a^*b)^*ba + ab).$$

$$\text{Similarly: } X = (b^*a(a^*b)^*b^*a + a^*b)^*(b^*a(a^*b)^*ab + ba).$$

The solution is unique, since $\varepsilon \notin L(a^*b)$ and $\varepsilon \notin L(b^*a)$.

System of linear equations on suffix languages

(Reminder:)

Let $A = (Q, \Sigma, \delta, q_0, F)$ be a deterministic finite automaton. The **suffix language** of A with respect to a state $q \in Q$ is

$$L(A, q) := \{v \in \Sigma^* \mid \delta(q, v) \in F\}.$$

$L(A, q)$ is the set of those words that would be accepted by A if the initial state were q .

So $L(A, q_0) = L(A)$ and $\varepsilon \in L(A, q) \iff q \in F$.

Furthermore, the following equations hold.

$$L(A, q) = \bigcup_{t \in \Sigma} t L(A, \delta(q, t)) \cup \begin{cases} \emptyset & \text{ha } q \notin F \\ \{\varepsilon\} & \text{ha } q \in F \end{cases}$$

where $tL := \{tu \mid u \in L\}$ ($t \in \Sigma$).

We solve this system of equations on suffix languages (focusing on $L(A, q_0)$). All coefficients are a single letter, so uniqueness follows from the previous theorem.

Regular expression for DFA

Example:

	a	b	c
$\rightarrow q_0$	q_1	q_2	q_3
q_1	q_4	q_2	q_4
q_2	q_0	q_4	q_3
$\leftarrow q_3$	q_4	q_3	q_4
q_4	q_4	q_4	q_4

Solution: The system of linear equations on suffix languages.

$$X := L(A, q_0), \quad Y := L(A, q_1),$$

$$Z := L(A, q_2), \quad V := L(A, q_3),$$

Observation: $L(A, q_4) = \emptyset$

$$X = aY + bZ + cV$$

$$Y = bZ$$

$$Z = aX + cV$$

$$V = bV + \varepsilon \quad X = ?$$

$$V = b^*, \quad Z = aX + cb^*, \quad Y = b(aX + cb^*),$$

$$X = ab(aX + cb^*) + b(aX + cb^*) + cb^*,$$

$$X = (aba + ba)X + (abc + bc + c)b^*,$$

$$X = (aba + ba)^*(abc + bc + c)b^*.$$

Generalized nondeterministic finite automata

Let $\mathcal{R}(\Sigma)$ be the set of regular expressions over Σ .

Definition

A **generalized nondeterministic finite automaton** (GNFA) is a quintuple, $A = (Q, \Sigma, \delta, Q_0, F)$, where

- ▶ Q is a finite, nonempty **set of states**,
- ▶ Σ is a finite alphabet of input symbols (**input alphabet**),
- ▶ $\delta : Q \times \mathcal{R}(\Sigma) \rightarrow \mathcal{P}(Q)$ is the **(state) transition function**,
- ▶ $Q_0 \subseteq Q$ is the **set of initial states** (or start states),
- ▶ $F \subseteq Q$ is the **set of accept(ing) states**.

So, a GNFA is like an NFA, but edges are labelled by regular expressions instead of letters in the transition diagram representation.

Generalized nondeterministic finite automata

Definition

Let $A = (Q, \Sigma, \delta, Q_0, F)$ be a generalized nondeterministic finite automaton and let $u, v \in Q\Sigma^*$ be configurations. A **reduces** u to v **in one step** (notation: $u \Rightarrow_A v$), if there are $q, p \in Q$, $z, w \in \Sigma^*$, $R \in \mathcal{R}(\Sigma)$, such that $p \in \delta(q, R)$, $z \in L(R)$, $u = qzw$ and $v = pw$ hold.

Multistep reduction and the language recognized by the automaton can be defined similarly to the definitions for NFA.

$u \Rightarrow_A^* v$, if $u = v$ or there is an integer $k \geq 1$ and configurations $w_0, \dots, w_k$ with $w_{i-1} \Rightarrow_A w_i$ ($1 \leq i \leq k$), $w_0 = u$ and $w_k = v$.

$L(A) = \{u \in \Sigma^* \mid q_0 u \Rightarrow_A^* p \text{ for some } q_0 \in Q_0 \text{ and } p \in F\}$

Generalized nondeterministic finite automata

Observation: GNFA $A = (Q, \Sigma, \delta, Q_0, F)$ accepts $u \in \Sigma^*$
 $\iff \exists n \in \mathbb{N}, u_1, \dots, u_n \in \Sigma^*, R_1, \dots, R_n \in \mathcal{R}(\Sigma), q_0 \in Q_0$ and $q_1, \dots, q_n \in Q$ with $u = u_1 \cdots u_n$, furthermore $u_i \in L(R_i)$, $q_i \in \delta(q_{i-1}, R_i)$ and $q_n \in F$ hold for all $1 \leq i \leq n$.

Observation: Considering the transition diagram representation of GNFA A , A accepts $u \in \Sigma^*$ if and only if there is a walk from some $q_0 \in Q_0$ to some state of F with edge labels $R_1, \dots, R_n$ with the property $u \in L(R_1 R_2 \cdots R_n)$.

Definition

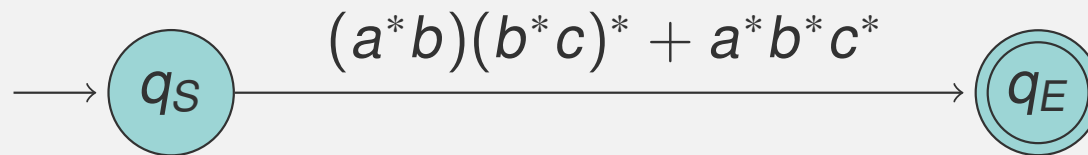
If all regular expressions in the transition function of GNFA $A = (Q, \Sigma, \delta, Q_0, F)$ is from $\Sigma \cup \{\varepsilon\}$ then A is called a **nondeterministic finite automaton with ε -moves** (ε NFA).

So, there are letter transitions, like in an NFA and ε -transitions (ε -moves) for which only the state is changed, but no letter is processed from the input.

DFA for regular expression

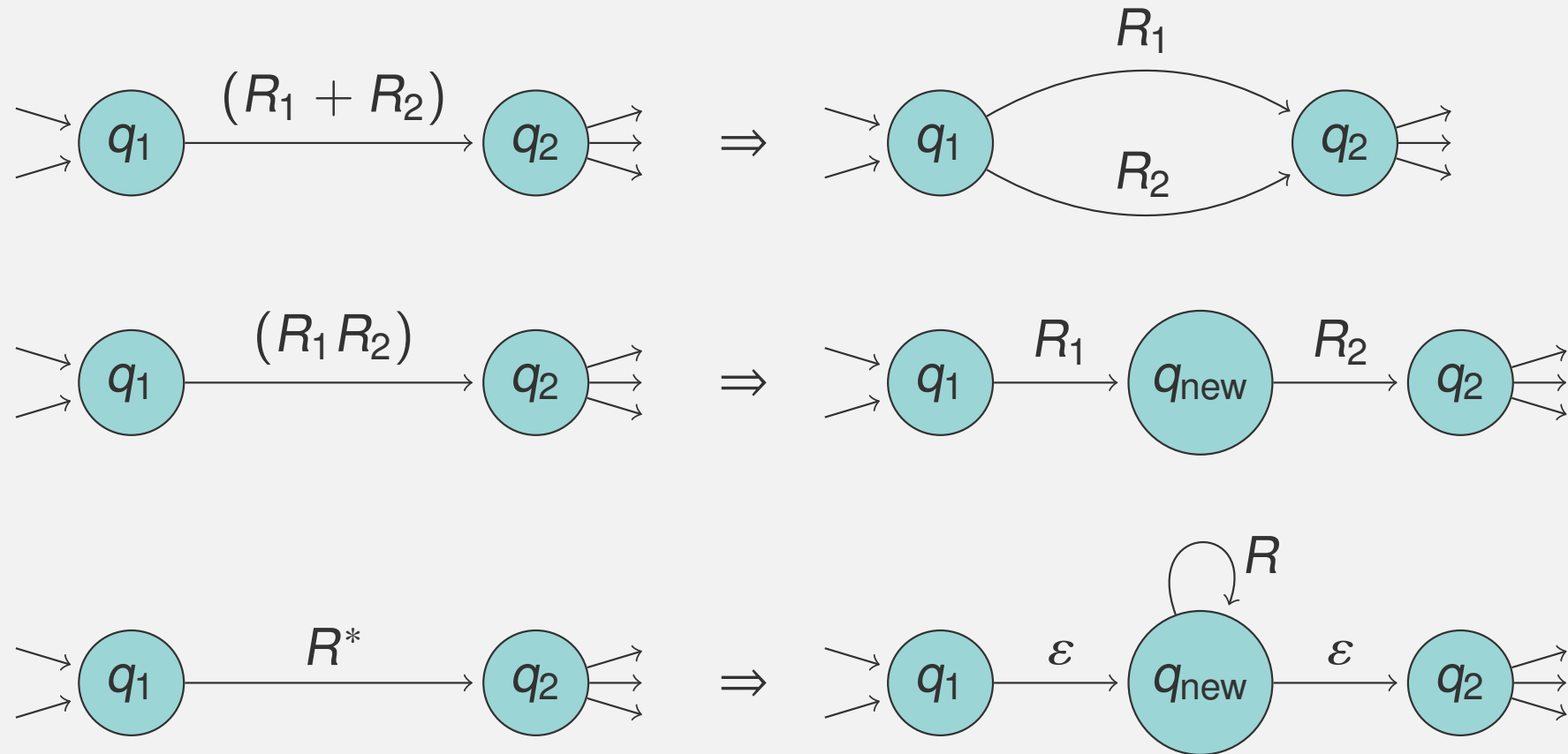
Exercise: Let $R = (a^*b)(b^*c)^* + a^*b^*c^*$ be a regular expression. Construct a DFA A with the property $L(A) = L(R)$.

Step 0 Given a regular expression R it is easy to construct a GNFA A with the property $L(A) = L(R)$. Let $A = (\{q_S, q_E\}, \Sigma, \delta, \{q_S\}, \{q_E\})$ and let $\delta(q_S, R) = \{q_E\}$ be its only transition. Obviously, $L(A) = L(R)$ holds.



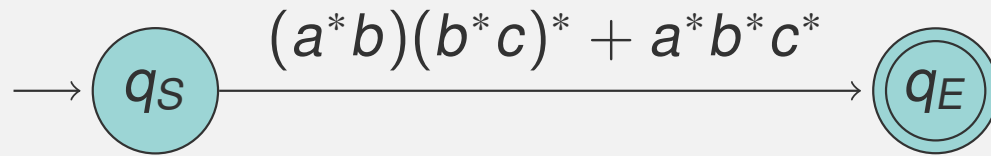
DFA for regular expression

Step 1 The following transformations do not change $L(A)$.

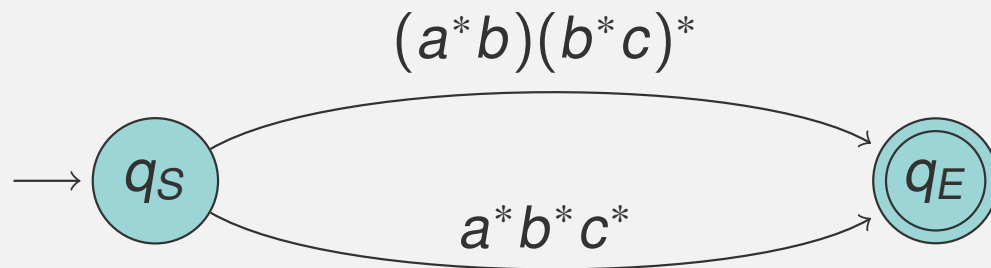


E.g., in the 3rd case, using any new edge implies the usage of (q_1, q_{new}) once, $(q_{\text{new}}, q_{\text{new}})$ k -fold ($k \geq 0$) and (q_{new}, q_2) once. This corresponds to 1-fold usage of the edge (q_1, q_2) labelled R^* in the automaton before the transformation.

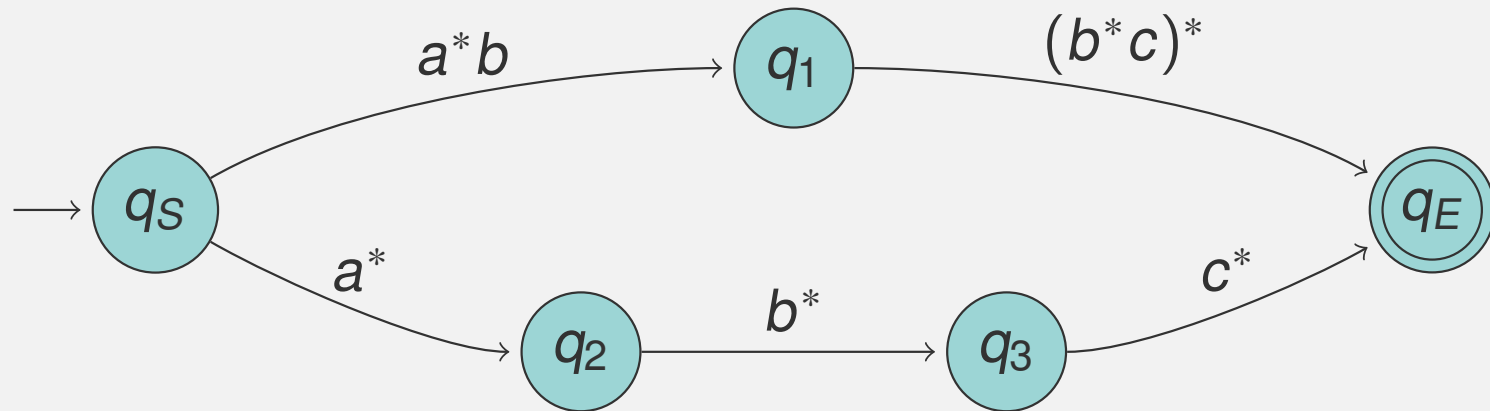
DFA for regular expression



$\Downarrow$

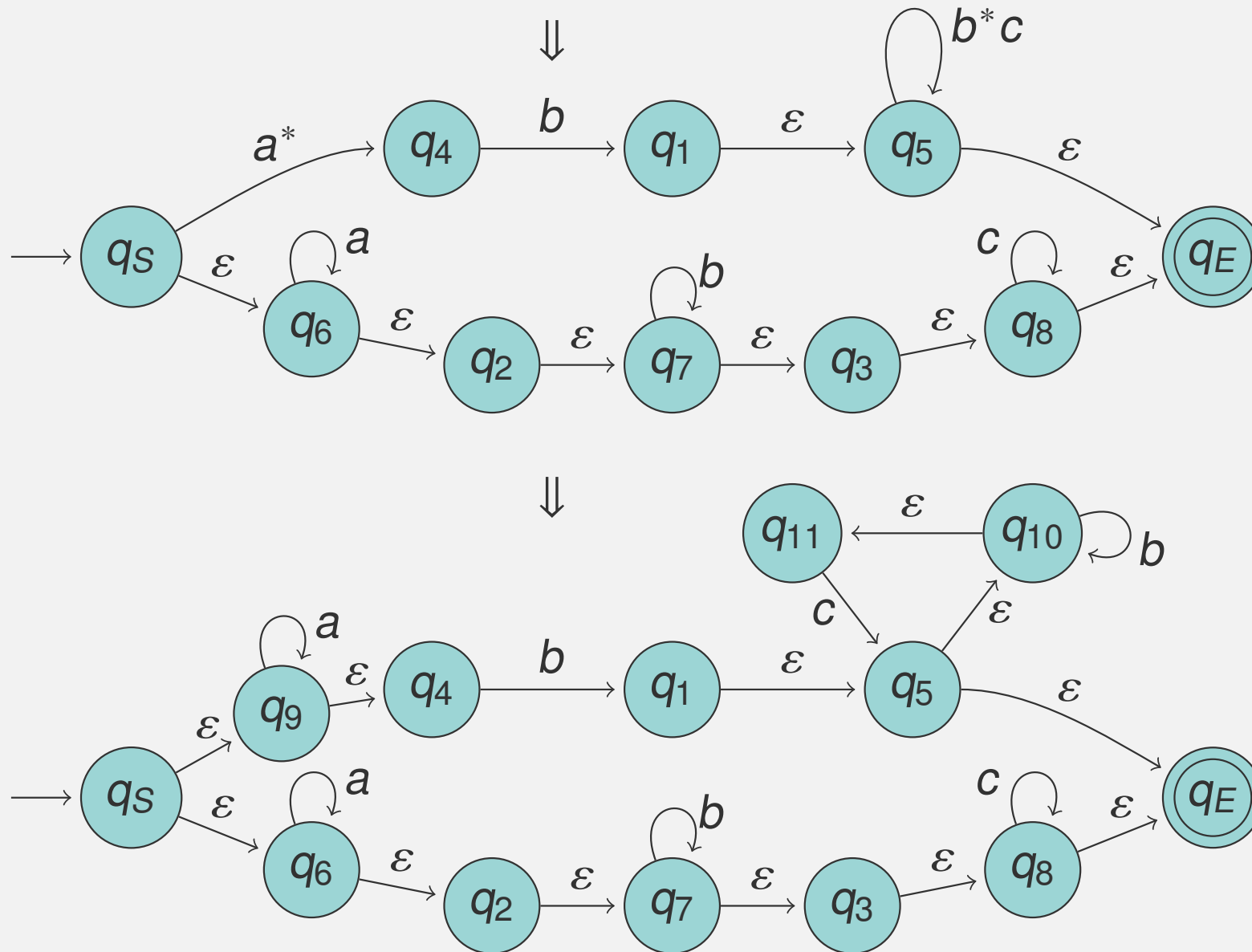


$\Downarrow$



$\Downarrow$

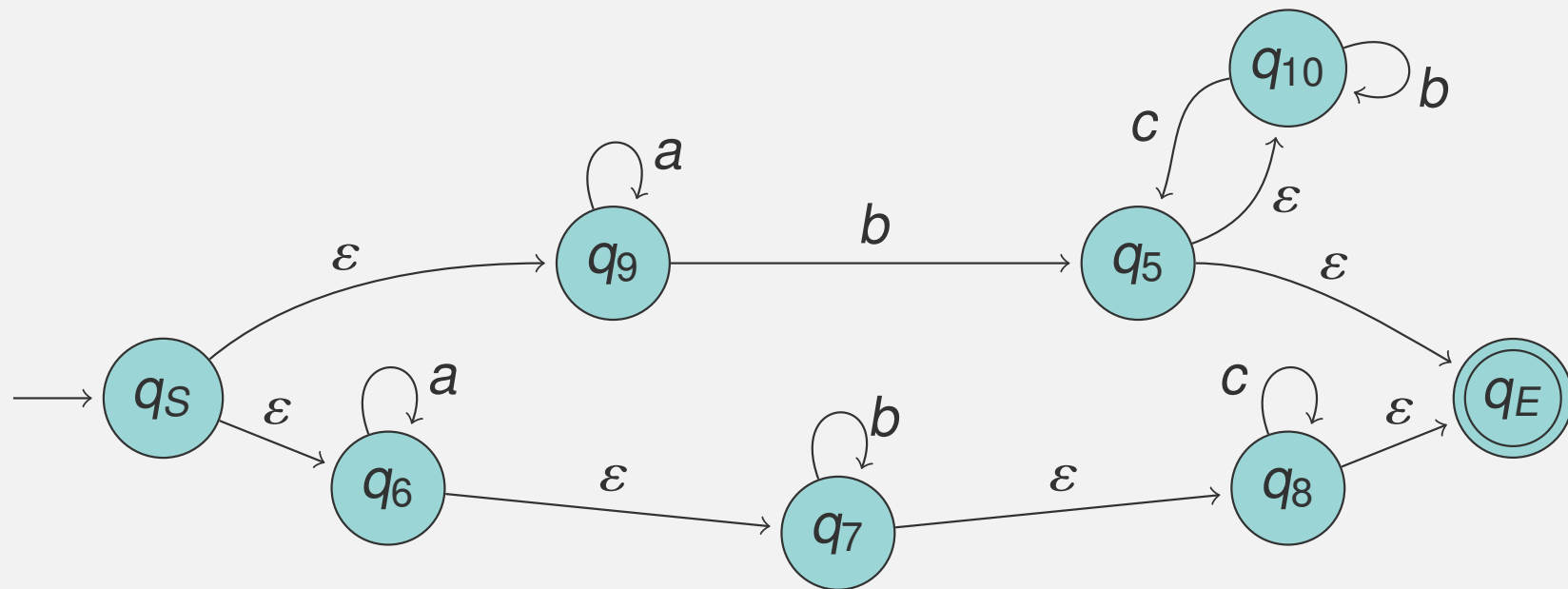
DFA for regular expression



DFA for regular expression

Simplification: If $\delta(q, t) = \{q'\}$ and $\delta(q', t') = \{q''\}$ ($t, t' \in \Sigma \cup \{\varepsilon\}$ and at least one of them is ε) are the **single** transitions to and from q' , respectively, then q' can be eliminated if the rule $q'' \in \delta(q, tt')$ is added to δ .

In our running example states $q_1, q_4, q_2, q_3, q_{11}$ can be eliminated.



DFA for regular expression

Step 2: ε NFA to NFA

Let $A = (Q, \Sigma, \delta, Q_0, F)$ be an ε NFA. First, for every $q \in Q$ the set of states $H(q)$ reachable from q by ε -moves is determined.

I.e., $H(q) = \{q' \in Q \mid q \Rightarrow_A^* q'\}$.

$H_0(q) := \{q\}$. $H_{i+1}(q) := H_i(q) \cup \{q' \in Q \mid \exists r \in H_i(q) : r \Rightarrow_A q'\}$.

$H_0(q) \subseteq H_1(q) \subseteq \dots \subseteq Q$.

This sequence becomes stable after finite steps, the stable set equals to $H(q)$. (Only states, that are reachable from q by ε -moves become an element of $H_i(q)$ for all $i \in \mathbb{N}$, and all such states become a member of $H_i(q)$ for some $i \in \mathbb{N}$.)

NFA $A' = (Q, \Sigma, \delta', Q_0, F')$ is defined as follows.

$r \in \delta'(q, t) \iff \exists q' \in H(q) : r \in \delta(q', t) \quad (q \in Q, t \in \Sigma)$

$F' := \{q \in Q \mid H(q) \cap F \neq \emptyset\}$.

DFA for regular expression

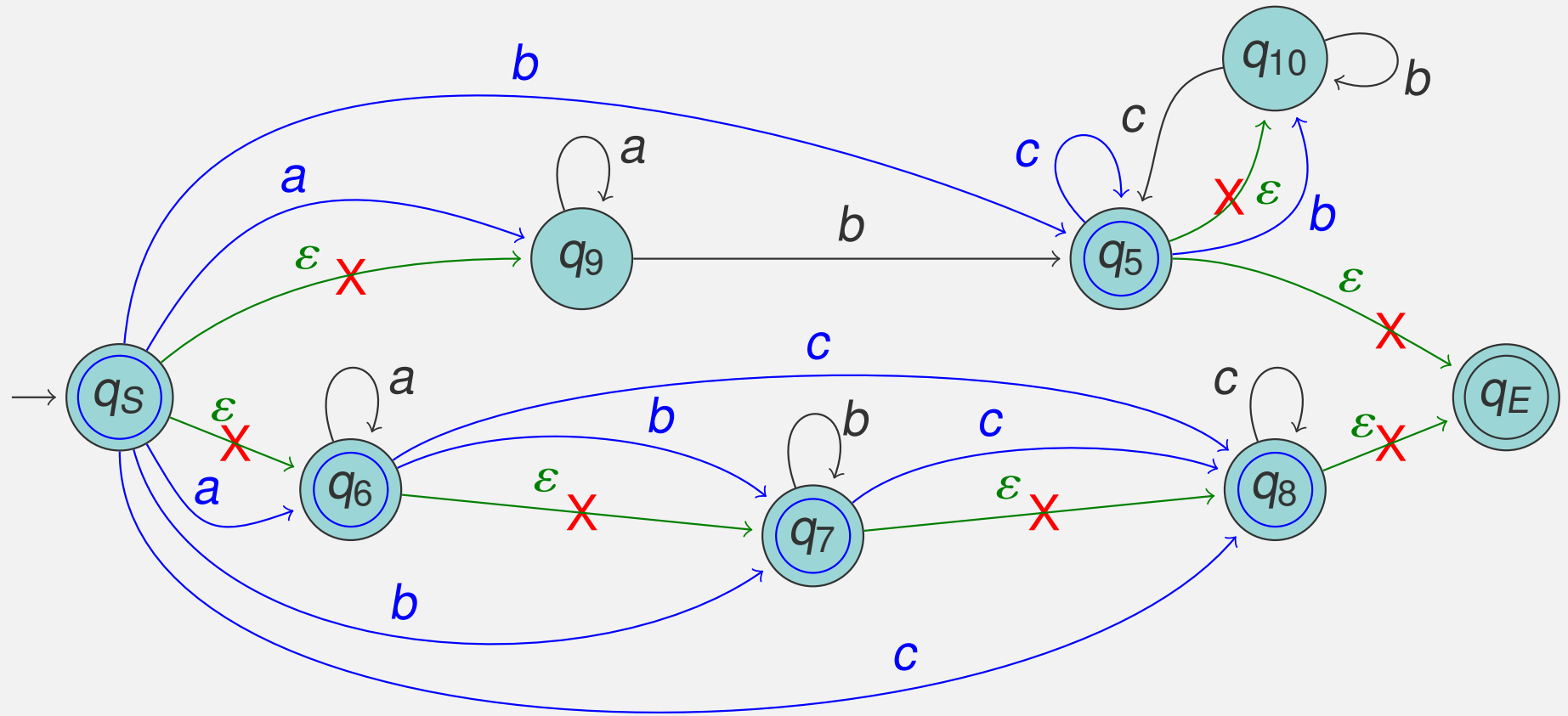
$L(A') \subseteq L(A)$, since applying a new rule $r \in \delta'(q, t)$ corresponds to a reduction $q \Rightarrow_A^* q'$ followed by an application of $r \in \delta(q', t)$ for some $q' \in H(q)$.

On the other hand, new rules make ε -moves unnecessary as they replace any sequence of ε -moves followed by a rule for some $t \in \Sigma$.

It can happen, that a reduction ends with a sequence on ε -moves. But these ε -moves at the end of the reductions are unnecessary as well due to the extension of F to F' .

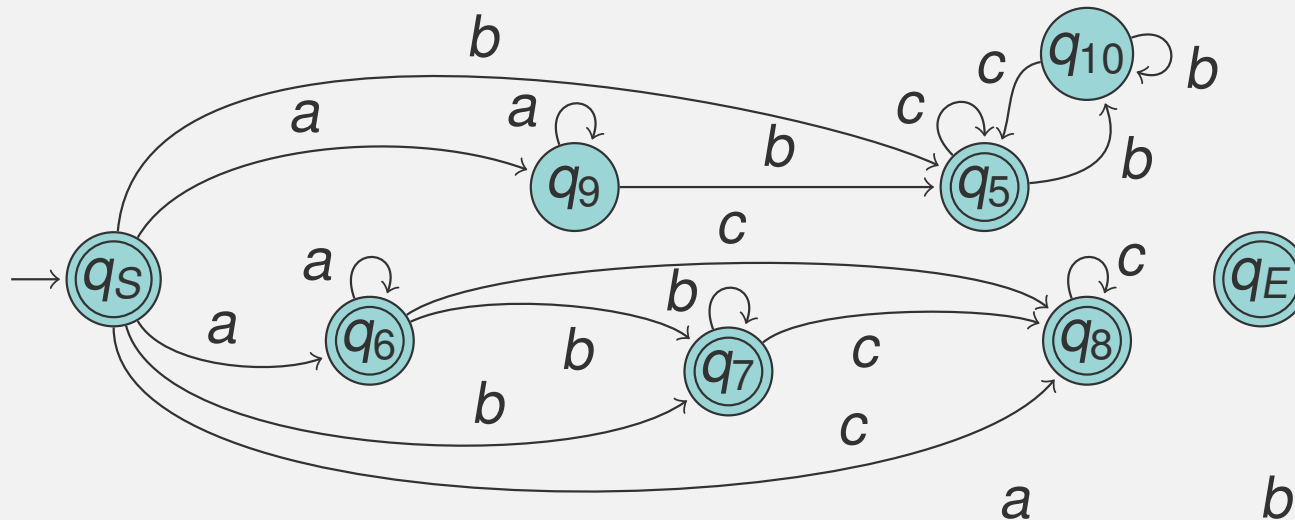
So $L(A) \subseteq L(A')$ holds, too.

DFA for regular expression



DFA for regular expression

Step 3 NFA to DFA



		a	b	c
$\Leftrightarrow$	$\{q_5\}$	$\{q_6, q_9\}$	$\{q_5, q_7\}$	$\{q_8\}$
$\leftarrow$	$\{q_6, q_9\}$	$\{q_6, q_9\}$	$\{q_5, q_7\}$	$\{q_8\}$
$\leftarrow$	$\{q_5, q_7\}$	$\{\}$	$\{q_7, q_{10}\}$	$\{q_5, q_8\}$
$\leftarrow$	$\{q_8\}$	$\{\}$	$\{\}$	$\{q_8\}$
	$\{\}$	$\{\}$	$\{\}$	$\{\}$
$\leftarrow$	$\{q_7, q_{10}\}$	$\{\}$	$\{q_7, q_{10}\}$	$\{q_5, q_8\}$
$\leftarrow$	$\{q_5, q_8\}$	$\{\}$	$\{q_{10}\}$	$\{q_5, q_8\}$
	$\{q_{10}\}$	$\{\}$	$\{q_{10}\}$	$\{q_5\}$
$\leftarrow$	$\{q_5\}$	$\{\}$	$\{q_{10}\}$	$\{q_5\}$

A pumping lemma for regular languages

Lemma

For every language $L \in \mathcal{L}_3$ there is a constant $n \in \mathbb{N}$, such that for every word $w \in L$ and subwords u, v, w' of w with the properties $w = uw'v$ $|w'| \geq n$ there exist subwords x, y, z of w' with the properties $w' = xyz$, $0 < |y| \leq n$ and $uxy^i zv \in L$ for all $i \geq 0$.

Less formally: Every long enough subword of any member of L has a short, but nonempty subword with the property, that no matter how many times we iterate ("pump") this subword, the resulted word is in L .

Proof:

Since $L \in \mathcal{L}_3$ there exists a deterministic finite automaton $A = (Q, \Sigma, \delta, q_0, F)$ with $L(A) = L$. Let $n = |Q|$.

Consider a word $w \in L$ and words $u, w', v \in \Sigma^*$ with the properties $w = uw'v$ and $|w'| \geq n$. Let $w' = t_1 \cdots t_m$, where $t_i \in \Sigma$ ($1 \leq i \leq m$) and $m \geq n$.

A pumping lemma for regular languages

Then $q_0 w \Rightarrow_A^* r_0 t_1 t_2 \cdots t_m v \Rightarrow_A r_1 t_2 \cdots t_m v \Rightarrow_A \cdots \Rightarrow_A r_m v \Rightarrow_A^* p$
holds for some states $r_0, \dots, r_m$ and $p \in F$.

Since we have $n = |Q| < m + 1$ there exists natural numbers k, j with $0 \leq j < k \leq n$ and $r_j = r_k$ (due to pigeonhole principle).

Let $x = t_1 \cdots t_j$, $y = t_{j+1} \cdots t_k$, $z = t_{k+1} \cdots t_m$. By the choice of j and k we have $0 < |y| \leq n$.

The only thing to prove is that for all $i \geq 0$ we have $uxy^i zv \in L$.

We claim, that $r_j y^i \Rightarrow_A^* r_j$ holds for all $i \geq 0$.

This can be proved easily by induction. For $i = 0$ we have $r_j \Rightarrow_A^* r_j$,
for $i = 1$ we have $r_j y \Rightarrow_A^* r_k = r_j$, for greater i 's we have
 $r_j y^i \Rightarrow_A^* r_j y^{i-1} \Rightarrow_A^* r_j$.

So for all $i \geq 0$ we have

$q_0 uxy^i zv \Rightarrow_A^* r_j y^i zv \Rightarrow_A^* r_j zv = r_k zv \Rightarrow_A^* p$ implying $uxy^i zv \in L$.

A pumping lemma for regular languages

Example: Let $L = \{u \in \{a, b\}^* \mid u = u^R\}$ be the language of palindromes over $\{a, b\}$.

Assume, on the contrary, that $L \in \mathcal{L}_3$. Let $w = a^n b a^n$ where n is the constant of the pumping lemma for L . As w is a palindrome, we have $w \in L$. Let $u = \varepsilon$, $w' = a^n$, $v = b a^n$.

Since $|w'| \geq n$, the pumping lemma can be applied. So, there exists a decomposition $w' = xyz$ of w' with the properties $0 < |y| \leq n$ and $uxy^i z v \in L$ for all $i \geq 0$.

By the choice of w and the conditions on y we have $y = a^k$ for some $0 < k \leq n$, so $uxy^0 z v = ux z v = a^{n-k} b a^n \notin L$.

So we have $uxy^0 z v \in L$ and $uxy^0 z v \notin L$, leading to a contradiction. It means, that our original assumption, $L \in \mathcal{L}_3$ should be false.